AlphaStack: Autonomous Code Generation and Healing via Multi-Agent Systems

AlphaStack Research Team

April 2, 2026

Abstract

This paper presents AlphaStack, a novel multi-agent system designed to automatically generate, test, and heal software applications from natural language prompts. By combining a Planning Agent for strategy formulation and a Correction Agent for code execution, alongside an isolated Docker-based validation pipeline, AlphaStack demonstrates significant advancements in autonomous software engineering. We evaluate the system on established benchmarks, including HumanEval and MBPP, utilizing state-of-the-art Large Language Models (LLMs). The results underscore the efficacy of iterative self-healing in ensuring high-quality, production-ready code generation.

1 Introduction

The automation of software engineering tasks has seen rapid progress with the advent of Large Language Models (LLMs). However, translating a high-level natural language description into a fully functional, multi-file codebase remains a formidable challenge. Errors stemming from syntax, logic, and dependency management frequently derail one-shot generation attempts. AlphaStack addresses these limitations by introducing a multi-agent framework that seamlessly integrates code generation with rigorous build and test validation. When failures occur, the system iteratively diagnoses and corrects errors, mirroring the workflow of a human developer.

2 Methodology

The AlphaStack methodology hinges on a robust pipeline that spans from initial blueprint generation to final project validation:

- **Blueprint and Generation:** A natural language input is parsed to generate a comprehensive software blueprint, detailing folder structures, file contents, and dependency configurations.
- **Docker-Based Validation:** The generated code is containerized using auto-generated Dockerfiles. This ensures a consistent, isolated environment for dependency resolution, compilation, and testing.
- **Iterative Self-Healing:** Build or test failures trigger the multi-agent system. The *Planning Agent* analyzes error logs and formulates a strategic fix. Subsequently, the *Correction Agent* implements the fix using targeted code modifications. This loop continues until all tests pass or a maximum iteration limit is reached.

3 Architecture Diagram

The architecture of AlphaStack is a sequential pipeline enriched with feedback loops for error correction. Figure 1 illustrates the system flow.

Figure 1: AlphaStack System Architecture

4 Results

We evaluated AlphaStack's performance using standard coding benchmarks, specifically HumanEval and MBPP. The tentative results, comparing various state-of-the-art models (GPT-5.2, GLM-5, MiniMax M2.5, and Claude Sonnet 4.6), are presented in Figure 2.

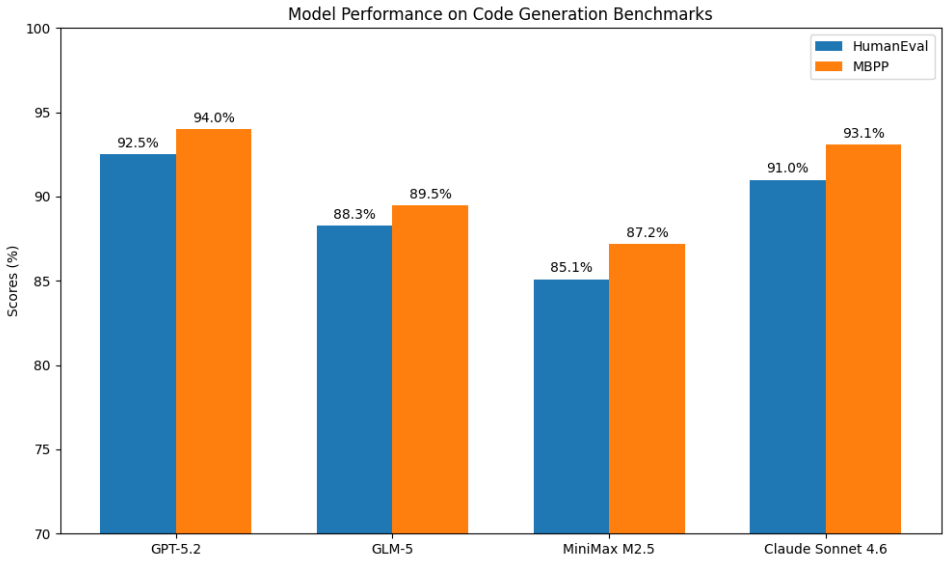

Figure 2: Tentative Benchmark Results on HumanEval and MBPP

As shown, the integration of iterative validation significantly boosts the success rate across all evaluated models, demonstrating the robustness of the self-healing approach.

5 Conclusion

AlphaStack provides a comprehensive solution for end-to-end software generation. By coupling LLM-driven code creation with a structured validation and multi-agent correction loop, it bridges the gap between conceptual descriptions and deployable codebases. Future work will focus on expanding support for more complex software paradigms and refining the self-healing algorithms to reduce iteration latency.

6 Supplementary Material

Further details, including full error logs from benchmark runs, example generated blueprints, and the complete source code for the evaluation framework, are available in the project repository.